2.

Under the Hood: Deconstructing Large Language Models

2.1

Overview

Tokens

75 words ≃ 100 tokens

What is the capital city of France ?

Tokens	Characters
7	34

What is the capital city of France ?

Token calculator

Parameters
System prompt: Answer in French
Temperature: 0.2
Max tokens: 7

Paris est la capitale de la

Tokens	Characters
7	28

Paris est la capitale de la

Tokens

create a photo of a llm as a judge

Image created

llama

llm

[680, 2152, 198, 680, 76]

Basics

Language Models: probability distribution over a sequence of words $p(x_1, \dots x_n)$

P(Transformers, are, encoder, decoder, models) = 0.01
P(Transformers, are, are, encoder, decoder, models) = 0.0001 *Syntactic knowledge*
P(Transformers, are, decoder, models) = 0.001 *Semantic knowledge*

Autoregressive language models:

The chain rule of probability: $P(x_1, x_{,2}, \dots, x_n) = P(x_1)\, P(x_2/ x_1)\, P(x_3/x_2,x_1) \dots$

P(Transformers, are, encoder, decoder, models) =

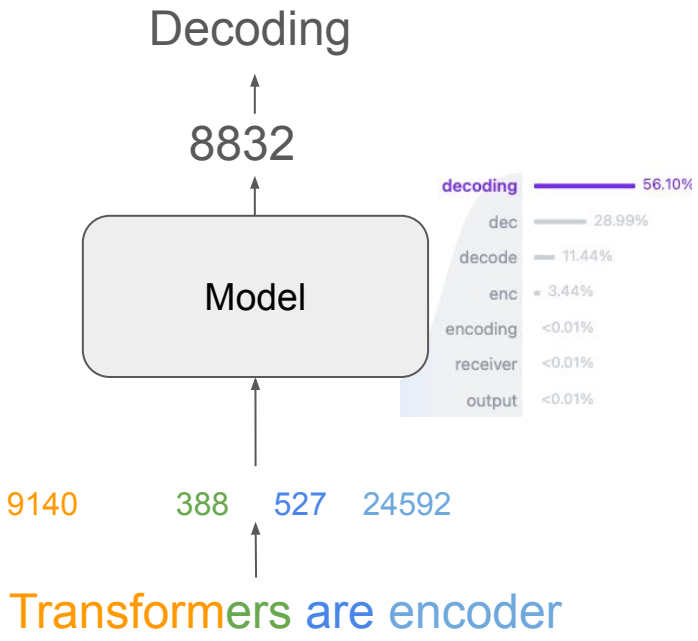
P (Transformers)
. P (Transformers, are / Transformers)
…
. P (models /Transformers, are, encoder, decoder)

Prediction process

The goal is to generate token by token

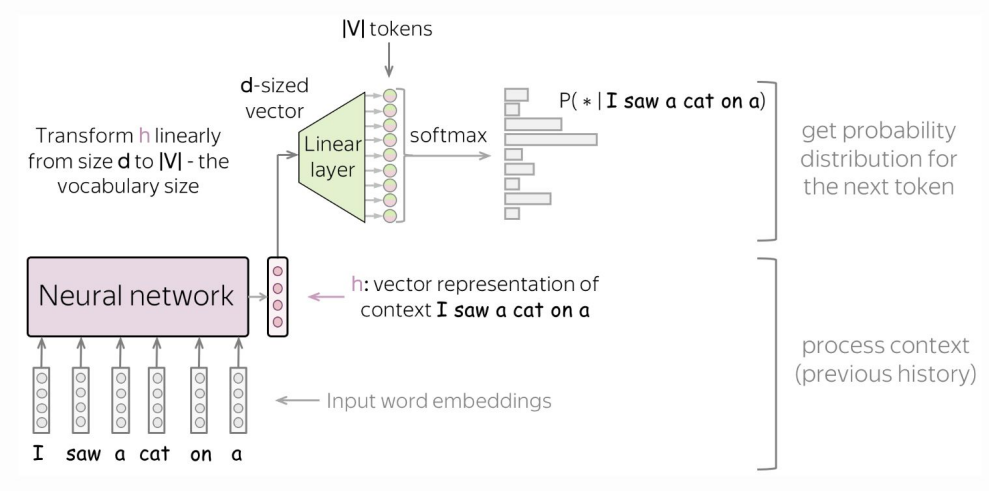
The steps for generation:

1. Tokenize
2. Feed the model with the token
3. Predict the probability of each possible token
4. Sample from the likelihood
5. Detokenize



Prediction process

1. Feed **word embedding** for previous (context) words into a network
2. Get **vector representation** of context from the network
3. From this vector representation, **predict** a probability distribution for the next token.



Lena Voita, Language Modeling [Blog]

Example

F Choose a random number between 1 and 50

The user is asking me to choose a random number between 1 and 50. 1s

27

Choose a random number between 1 and 50

Sure! Here's a random number between 1 and 50:

27 🎲

Choose a random number between 1 and 50

Show code

Show thinking

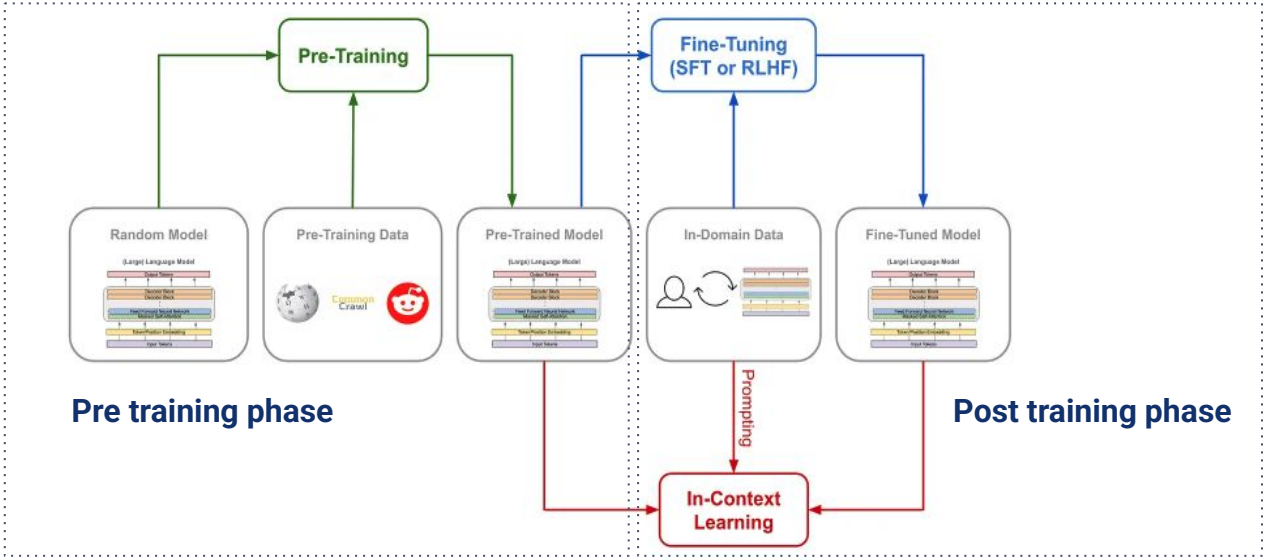
The random number is 13.

Show thinking

Generating a Number

I've decided on the `random` *module in Python. Seems like the most straightforward way to produce the desired integer. I'll import it and then use* `random.randint(1, 50)` *to get the number.*

Training Phases



Different stages of training for an LLM

2.2

Pre-training

Pre-training Phases

Data Preprocessing
Use all internet as data
Clean the data

↓

Tokenization
Turn data in numbers

↓

Training

Transformers
Choose a model for text generation

Data Preprocessing

Idea: use all of the clean internet
Note: internet is dirty & not representative of what we want.

Practice:

1. **Download all of internet**. *Common crawl: 250 billion pages, > 1PB (>1e6 GB)*
2. **Text extraction from HTML** (challenges: math, boilerplate)
3. **Filter undesirable content** (e.g. NSFW, harmful content, PII)
4. **Deduplicate** (url/document/line). *E.g. all the headers/footers/menu in forums are always same*
5. **Heuristic filtering**. *Remove low quality documents (e.g. # words, word length, outlier tokens, dirty tokens)*
6. **Model based filtering**. *Predict if page could be references by Wikipedia.*
7. **Data mix**. Classify data categories (code/books/entertainment). *Reweight domains using scaling laws to get high downstream performance.*

At the end of training, **overfit the model** on very good quality data

Tokenization

Byte-Pair Encoding (BPE)

```python
# !pip install tiktoken
import tiktoken

sentence = "Transformers are encoder decoder models"

tokenizer = tiktoken.get_encoding("cl100k_base")
tokens = tokenizer.encode(sentence)

print("Tokens:", tokens)
print("Decoded Tokens:", [tokenizer.decode([token]) for token in tokens])


sentence = "Une phrase en Francais moins optimisée"
tokenizer = tiktoken.get_encoding("cl100k_base")
tokens = tokenizer.encode(sentence)

print("-" * 10)
print("Tokens:", tokens)
print("Decoded Tokens:", [tokenizer.decode([token]) for token in tokens])

sentence = "12433 12 43"
tokenizer = tiktoken.get_encoding("cl100k_base")
tokens = tokenizer.encode(sentence)

print("-" * 10)
print("Tokens:", tokens)
print("Decoded Tokens:", [tokenizer.decode([token]) for token in tokens])
```

```
Tokens: [9140, 388, 527, 24592, 25569, 4211]
Decoded Tokens: ['Transform', 'ers', ' are', ' encoder', ' decoder', ' models']
----------
Tokens: [56948, 17571, 665, 31925, 936, 285, 40970, 7706, 285, 8047]
Decoded Tokens: ['Une', ' phrase', ' en', ' Fran', 'ca', 'is', ' moins', ' optim', 'is', 'ée']
----------
Tokens: [8874, 1644, 220, 717, 220, 3391]
Decoded Tokens: ['124', '33', ' ', '12', ' ', '43']
```

Tokenization

Tokens 189 Characters 907

Tokenization for LLMs (Large Language Models)

Tokenization is the process of breaking down text into smaller units called tokens before feeding it into a large language model. These tokens can be as short as a character or as long as a word or subword, depending on the tokenizer used.

LLMs don't process raw text directly—they work with numerical representations of tokens. For example, the sentence "I love AI" might be split into tokens like "I", " love", and " AI", each mapped to a unique number in the model's vocabulary.

Tokenization helps the model understand and generate language more efficiently, especially for languages with rich morphology or varied spelling. It also allows models to handle unknown or rare words by breaking them into recognizable sub parts.

In short, tokenization is a critical first step that transforms human-readable text into a format LLMs can process and learn from.

Text Token IDs

Tokens 346 Characters 1059

La tokenisation pour les LLM (Large Language Models)

La tokenisation est le processus qui consiste à découper un texte en unités plus petites appelées tokens avant de l'envoyer à un grand modèle de langage. Ces tokens peuvent etre aussi courts qu'un caractère ou aussi longs qu'un mot ou un sous-mot, selon le type de tokenizer utilisé.

Les LLM ne traitent pas directement le texte brut : ils utilisent des représentations numériques des tokens. Par exemple, la phrase « J'aime l'IA » peut ềtre découpée en tokens comme « J », «j'aime » et « l'IA », chacun étant associé à un identifiant unique dans le vocabulaire du modèle.

La tokenisation permet au modèle de comprendre et de générer du langage de manière plus efficace, notamment pour les langues riches en variations grammaticales ou orthographiques. Elle permet aussi de gérer les mots rares ou inconnus en les découpant en parties reconnues.

En résumé, la tokenisation est une étape essentielle qui transforme le texte lisible par l'humain en un format que les modèles peuvent traiter et apprendre.

Text Token IDs

GPT 3 Tokenizer

Tokens 179 Characters 907

Tokenization for LLMs (Large Language Models)

Tokenization is the process of breaking down text into smaller units called tokens before feeding it into a large language model. These tokens can be as short as a character or as long as a word or subword, depending on the tokenizer used.

LLMs don't process raw text directly—they work with numerical representations of tokens. For example, the sentence "I love AI" might be split into tokens like "I", " love", and " AI", each mapped to a unique number in the model's vocabulary.

Tokenization helps the model understand and generate language more efficiently, especially for languages with rich morphology or varied spelling. It also allows models to handle unknown or rare words by breaking them into recognizable sub parts.

In short, tokenization is a critical first step that transforms human-readable text into a format LLMs can process and learn from.

Text Token IDs

Tokens 225 Characters 1059

La tokenisation pour les LLM (Large Language Models)

La tokenisation est le processus qui consiste à découper un texte en unités plus petites appelées tokens avant de l'envoyer à un grand modèle de langage. Ces tokens peuvent etre aussi courts qu'un caractère ou aussi longs qu'un mot ou un sous-mot, selon le type de tokenizer utilisé.

Les LLM ne traitent pas directement le texte brut : ils utilisent des représentations numériques des tokens. Par exemple, la phrase « J'aime l'IA » peut être découpée en tokens comme « J », «j'aime » et « l'IA », chacun étant associé à un identifiant unique dans le vocabulaire du modèle.

La tokenisation permet au modèle de comprendre et de générer du langage de manière plus efficace, notamment pour les langues riches en variations grammaticales ou orthographiques. Elle permet aussi de gérer les mots rares ou inconnus en les découpant en parties reconnues.

En résumé, la tokenisation est une étape essentielle qui transforme le texte lisible par l'humain en un format que les modèles peuvent traiter et apprendre.

Text Token IDs

GPT 4 Tokenizer

Tiktoken
Open AI - Tokenizer

Model - Transformers

1. Parallelism: Model trainable of TB of data with GPU / TPU

2. Self-Attention: 12 288 questions asked for each word to understand its meaning

3. Positional Encoding: Most recent word might be more important that old ones

4. Multi-Head Attention: Parallelize step 2

5. Residual Connections: Limits the Vanishing Gradient problem

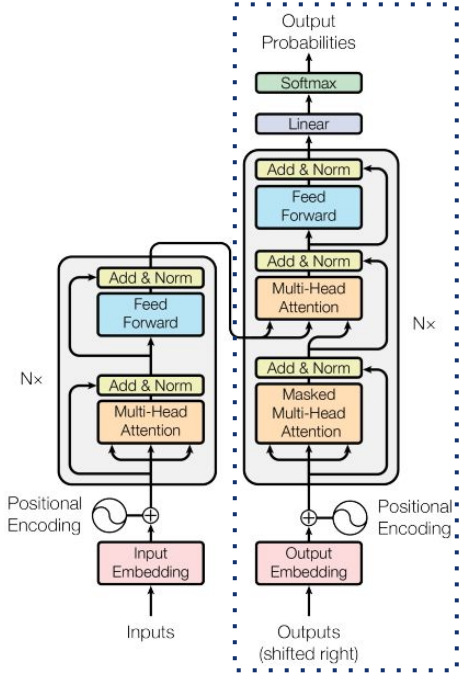
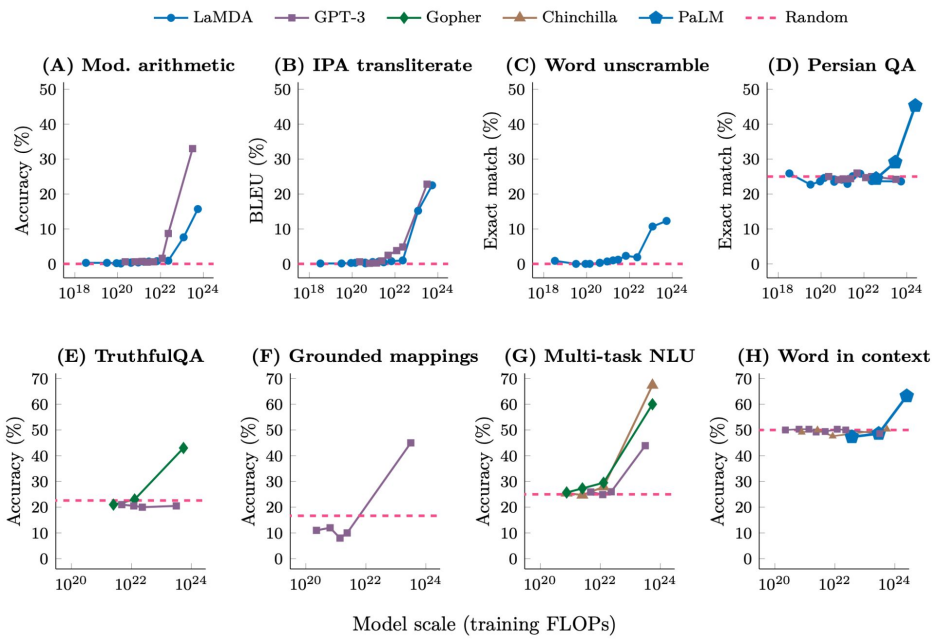


Figure 1: The Transformer - model architecture.

The Illustrated Transformer

Emergent abilities

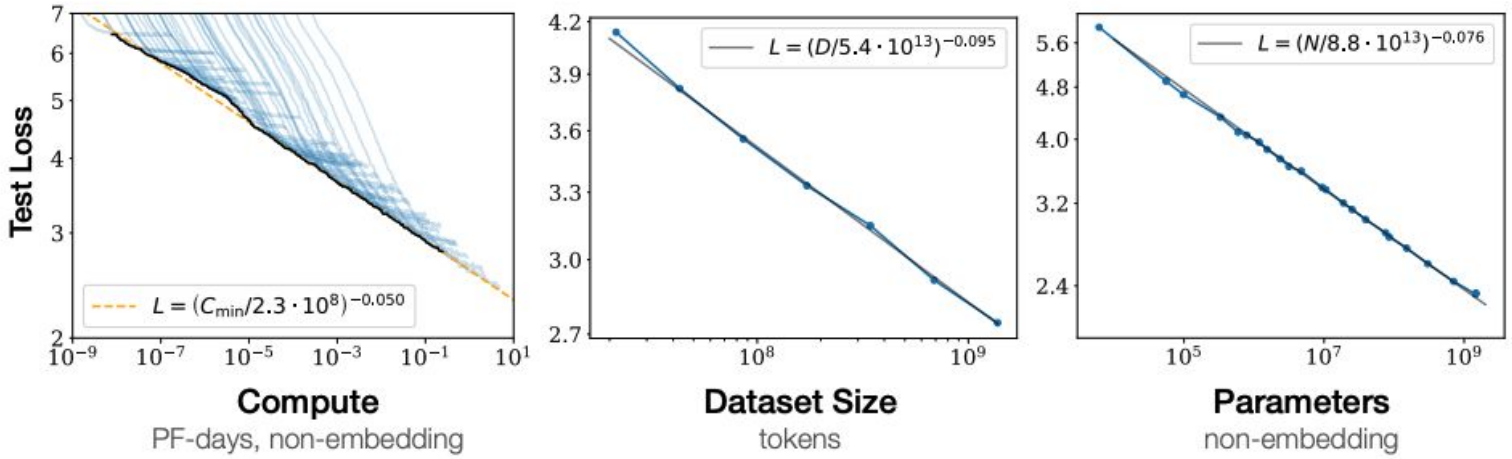
Models suddenly become good after enough training



Wei & Al, 2022, Emergent Abilities of Large Language Models

Scaling Laws

More resources, more data and bigger models -> better models



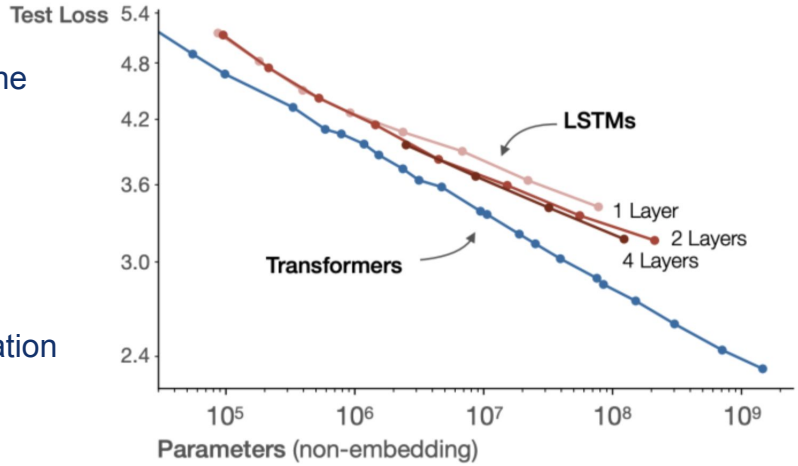
Jared Kaplan & Al, 2020, Scaling Laws for Neural Language Models

Transformer Hyper Parameters Optimization

Which architecture ?

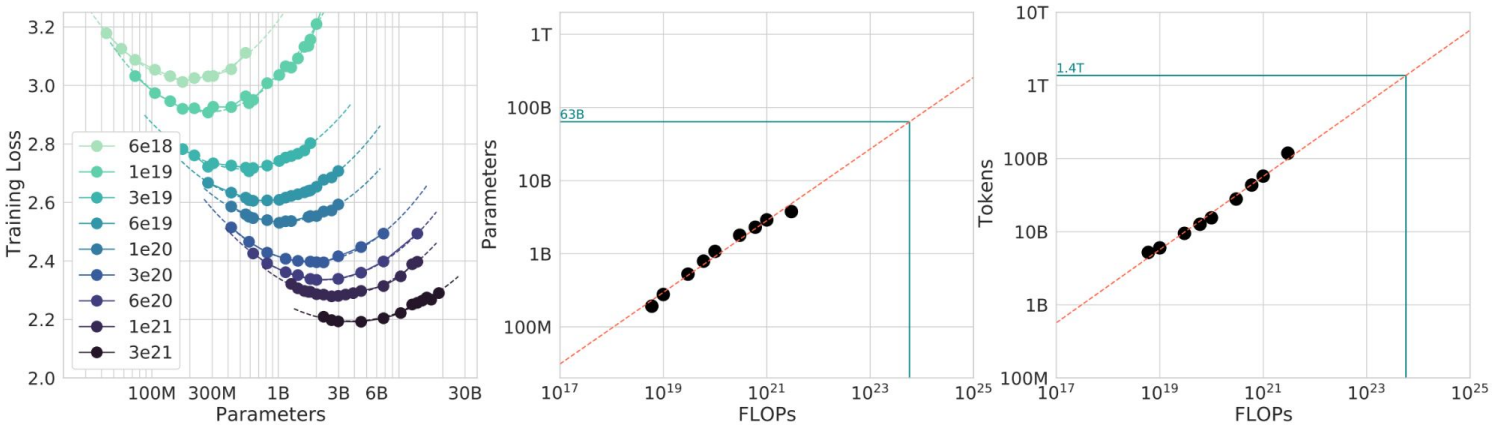
Steps

1. **Find** scaling recipes (example: learning rate decreases if the size of the model increases)
2. **Tune** hyper parameters on small models of different sizes
3. **Choose** the best models among the smallest ones
4. **Train** the biggest model with the best parameters configuration



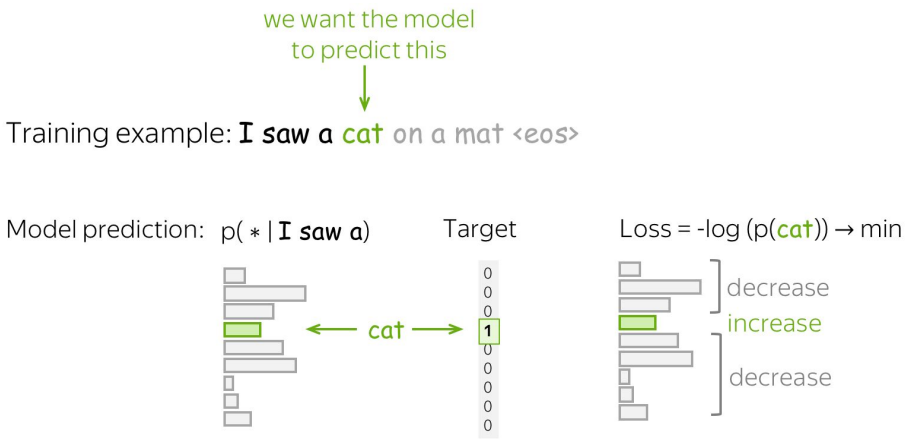
Chinchilla

How much compute, data & model parameters ?



Jordan Hoffmann & Al, 2023, Chinchilla, Training Compute-Optimal Large Language Models

Training



Lena Voita, Language Modeling [Blog]

Maximizing the likelihood is equivalent to **minimizing** the **cross entropy loss:**

$$\min \mathcal{L}(x_{i:L}) = \min \left(-\sum_i \log p(x_i \mid x_{i:i-1}) \right) = \max \prod_i p(x_i \mid x_{1:i-1})$$

Results

Prompt Why aren't birds real?

Completion GPT-3

Well, for one thing, The Rapture is not a very realistic event. There is no evidence that it is going to happen. I could say more, but you don't really need to know more.

Let's say I was a bird.

I've been a bird for a while now. I have never flown or seen another bird or any evidence of god. The rapture has never happened. I am a bird in a cage. I have never been a bird, but I am a bird.

InstructGPT

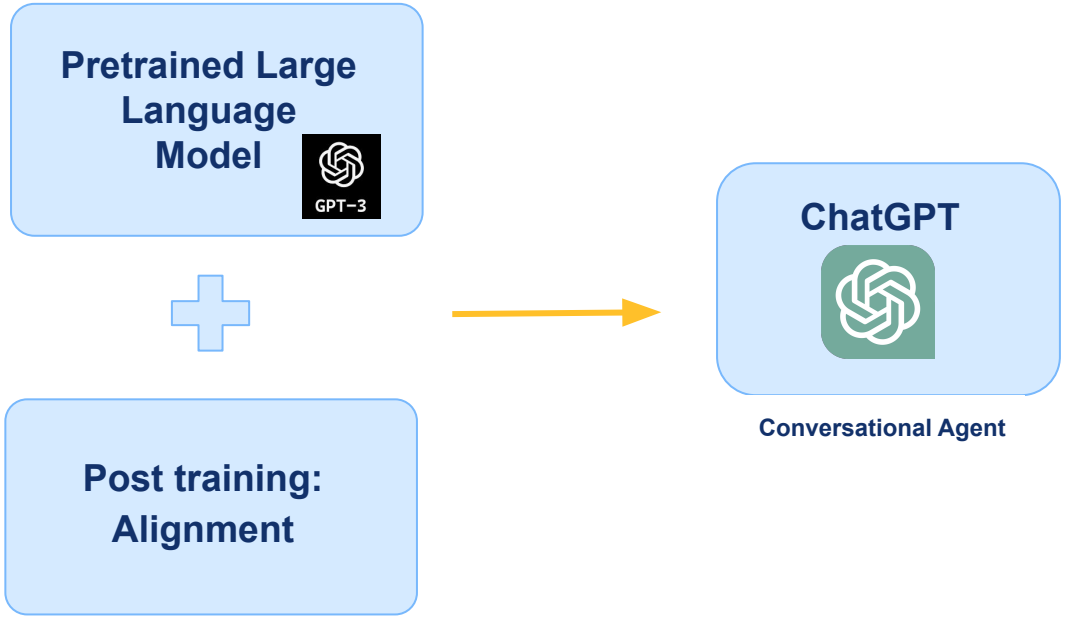
Birds are not real because they are not made of flesh and blood. They are made of feathers, bones, and organs.

Open AI, 2022, Aligning language models to follow instructions [Blog]

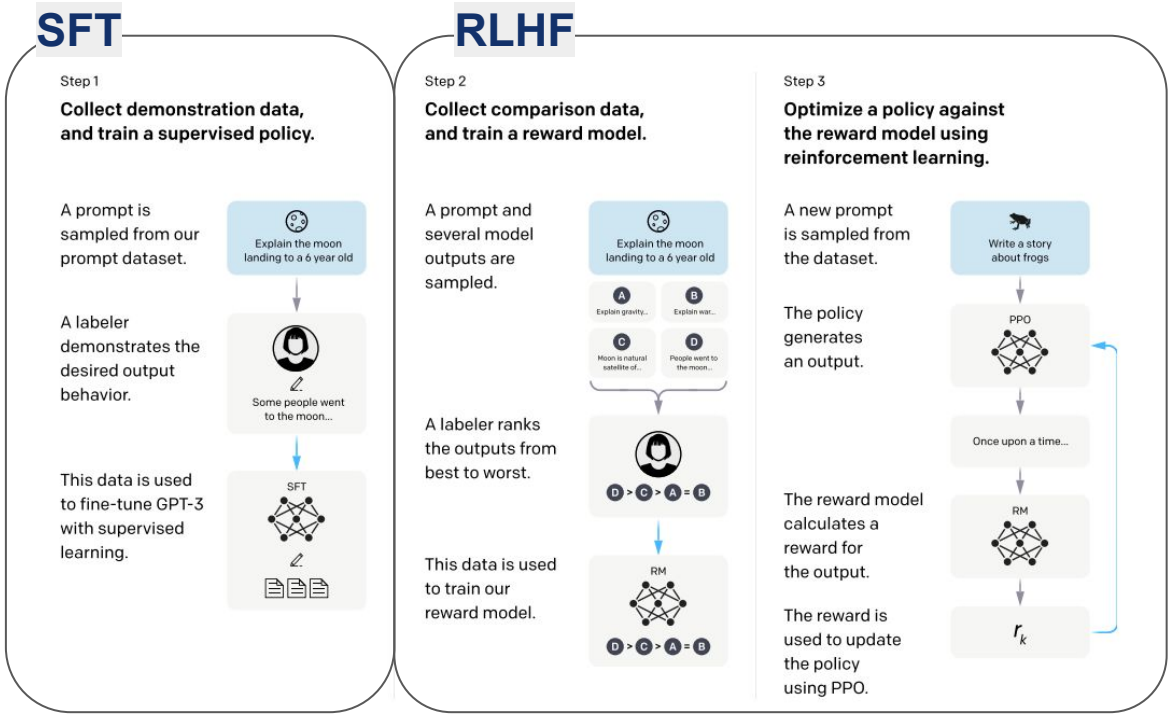
2.3

Post training

Post training - Alignment



Post training



2.3.1

Supervised Fined Tuning (SFT)

Supervise Fine Tuning (SFT)

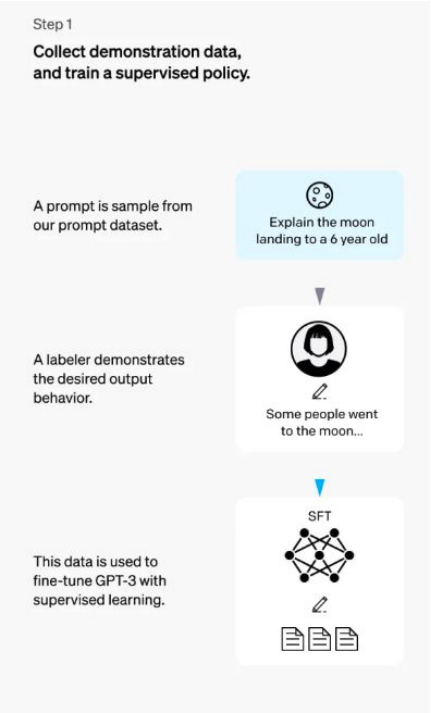
Prompt: Explain the moon landing to a 6 year old
Human labeller: Some people went to the moon

$$\mathcal{L}_{\text{SFT}} = -\sum_{i=1}^{N} \log\left(p_\theta(\text{Response}(i) \mid \text{Prompt}(i))\right)$$

Best SFT Use Cases:

Jumpstarting new model behavior:
- Pre-trained models -> Instruct Models
- Non-Reasoning models -> Reasoning models

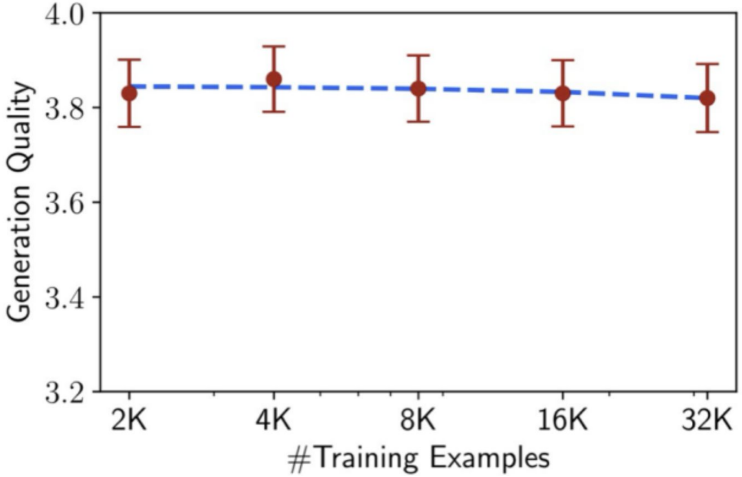
Improving model capabilities:
- Distilling capabilities for small models by training high-quality synthetic data generated from larger models



Open AI, 2022, Aligning language models to follow instructions [Blog]

Pros & Cons

Pros:



Cons:

- Behavior cloning
- Human abilities to answer perfectly to a given question
- Hallucination if answer from human not in training data
- Data collection cost

2.3.2

Reinforcement Learning from Human Feedback (RLHF)

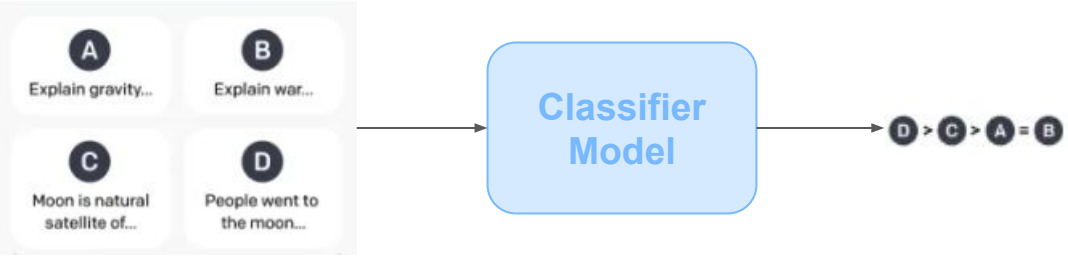
RLHF

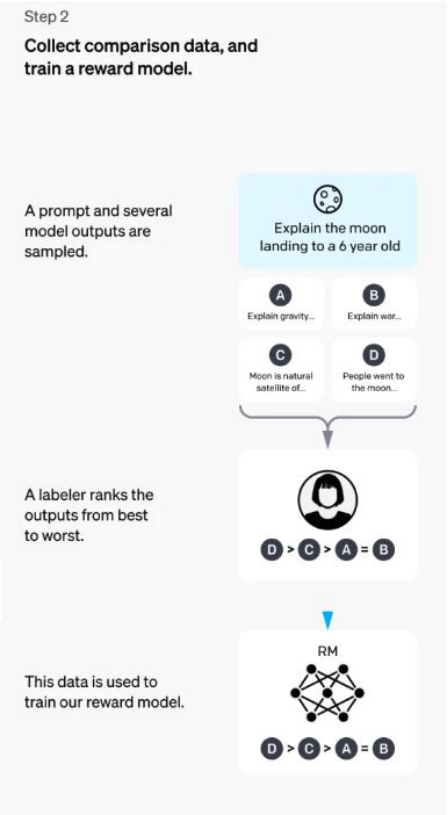
Step 1: Reward Model Training

Idea:

1. From a question, generate multiples answers
2. Ask a human to classify answers
3. Train a reward model to learn these preferences

Reward model: classifier that is trained to classify preferences from possibles answers





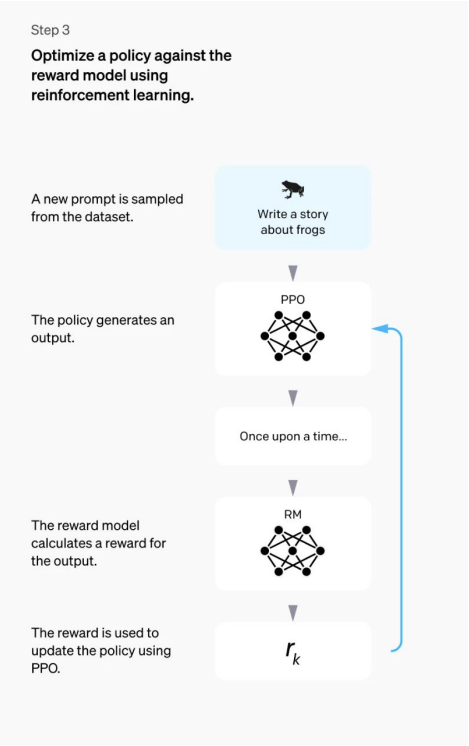
Open AI, 2022, Aligning language models to follow instructions [Blog]

Step 2: Proximal Policy Optimization (PPO)

“We then train a reward model (RM) on this dataset to predict which output our labelers would prefer.

Finally, we use this RM as a reward function and fine-tune our GPT-3 policy to maximize this reward using the PPO algorithm.”

Now, the new best policy is the **Direct Policy Optimization** (DPO).



2.3.3

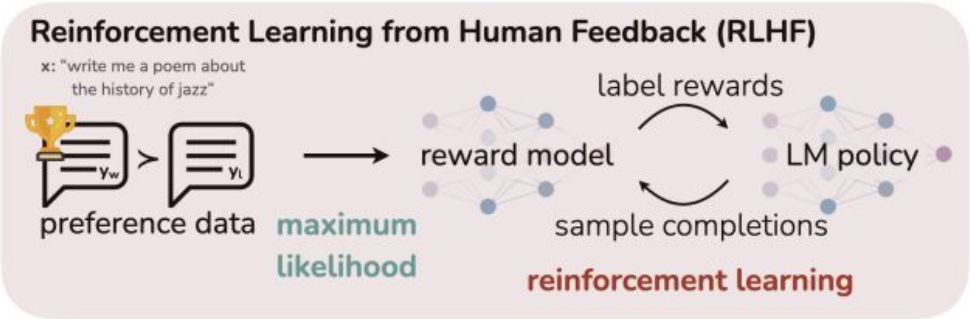
Direct Policy Optimization (DPO)

Direct Policy Optimization (DPO)

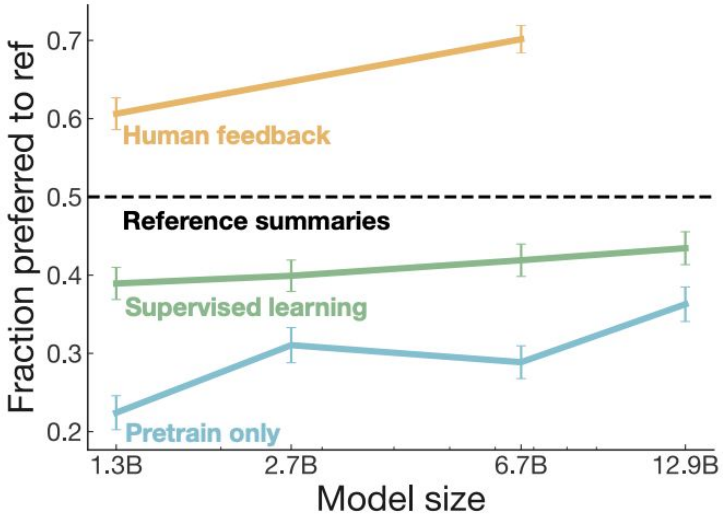
Prompt: Explain the moon landing to a 6 year old
Good response: Some people went to the moon
Bad response: Sorry, I am not able to answer

- PPO is much more complex (clipping, rollouts, outer loops) than in theory
- Maximize the desired output, minimize the other

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta\log\frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right]$$



Accuracy



Method	Simulated Win-rate (%)	Human Win-rate (%)
GPT-4*†	79.0 ± 1.4	69.8 ± 1.6
ChatGPT*†	61.4 ± 1.7	52.9 ± 1.7
PPO	46.8 ± 1.8	55.1 ± 1.7
DPO	46.8 ± 1.7	-
Best-of-1024	45.0 ± 1.7	50.7 ± 1.8
Expert Iteration	41.9 ± 1.7	45.7 ± 1.7
SFT 52k	39.2 ± 1.7	40.7 ± 1.7
SFT 10k	36.7 ± 1.7	44.3 ± 1.7
Binary FeedME	36.6 ± 1.7	37.9 ± 1.7
Quark	35.6 ± 1.7	-
Binary Reward Conditioning	32.4 ± 1.6	-
Davinci001*	24.4 ± 1.5	32.5 ± 1.6
LLaMA 7B*	11.3 ± 1.1	6.5 ± 0.9

Nisan Stiennon & Al, 2020, Learning to summarize from human feedback

Dubois* & Al, 2024, Alpaca Farm: A Simulation Framework for Methods that Learn from Human Feedback

33

Post training - Reasoning models

Open AI o1 features

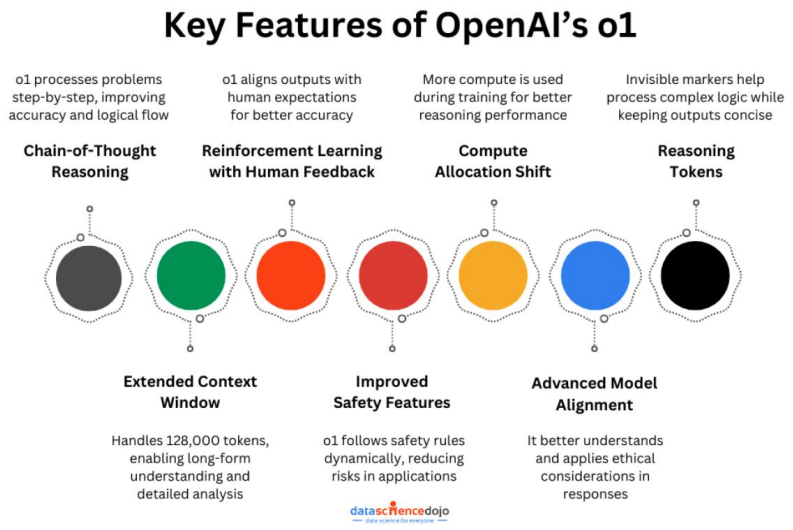
Idea: “o1 thinks before it answers”

Use Chain-of-thought reasoning (as seen previously) to fine-tune the model forcing him to output:

- The reasoning steps
- The answer

Longer but better answers

“more compute = better performance”



The Rise of Agentic Frameworks for Better Reasoning

Open AI o1 System Card

Introducing OpenAI o3 and o4-mini

Reasoning models

Example using Gemini 2.5 Pro

RLHF fined tuned LLM

provide an example answer with reasoning output for reasonning llm model

Show thinking

Seeking a Benchmark Puzzle

I'm currently scouting for a classic reasoning problem. I'm leaning toward a logic puzzle or multi-step math word problem, something to really put the reasoning LLM through its paces. Once I find a suitable challenge, I'll craft a prompt to see how it performs.

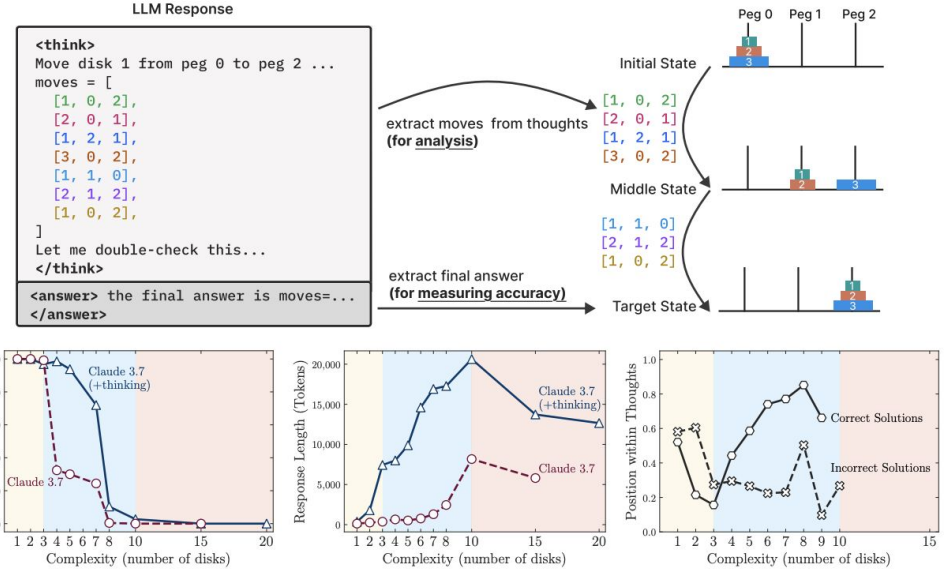
Formulating the Prompt

I've got the classic reasoning problem in mind, a real brain-teaser. Now, I'm shaping the prompt. It's crucial the reasoning LLM breaks down its logic step by step, separating the answer from the reasoning. This will serve as the perfect benchmark for "forced reasoning" assessment.

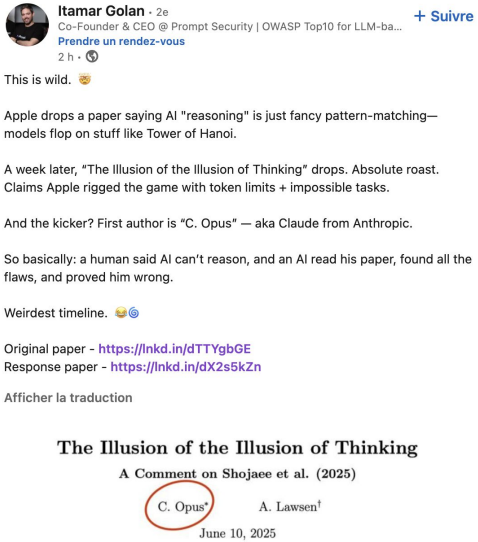
Of course. Here is an example of a typical reasoning problem and how a large language model (LLM) fine-tuned for "forced reasoning" would respond.

The key characteristic of the model's output is that it doesn't just give the final answer; it is compelled by its fine-tuning to first produce a structured, step-by-step logical process (its "reasoning") before concluding.

Reasoning war



The Illusion of Thinking



Itamar Golan · 2e
Co-Founder & CEO @ Prompt Security | OWASP Top10 for LLM-ba...
Prendre un rendez-vous
2 h ·
+ Suivre

This is wild. 😇

Apple drops a paper saying AI "reasoning" is just fancy pattern-matching—models flop on stuff like Tower of Hanoi.

A week later, "The Illusion of the Illusion of Thinking" drops. Absolute roast. Claims Apple rigged the game with token limits + impossible tasks.

And the kicker? First author is "C. Opus" — aka Claude from Anthropic.

So basically: a human said AI can't reason, and an AI read his paper, found all the flaws, and proved him wrong.

Weirdest timeline. 😂🌀

Original paper - https://lnkd.in/dTTYgbGE
Response paper - https://lnkd.in/dX2s5kZn

Afficher la traduction

The Illusion of the Illusion of Thinking

A Comment on Shojaee et al. (2025)

C. Opus* A. Lawsen†

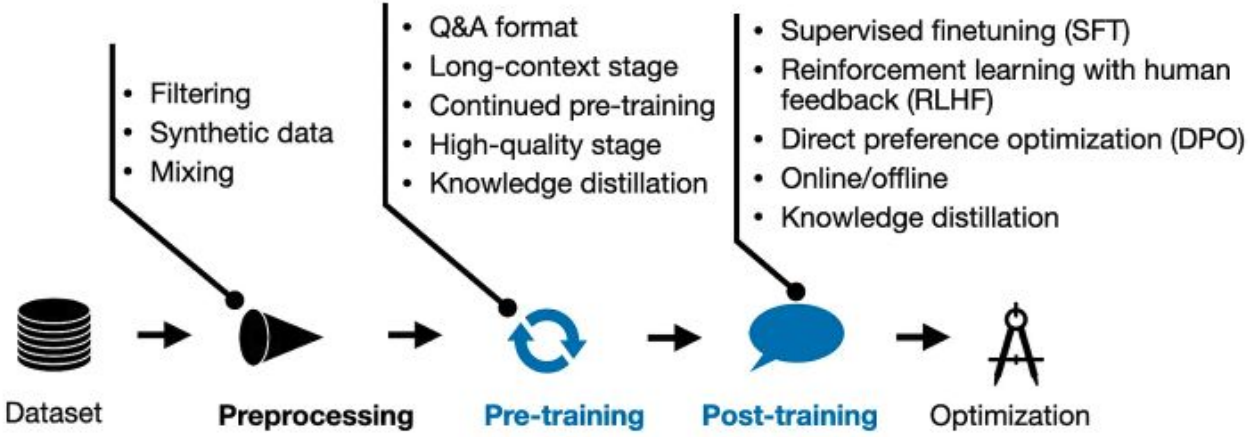
June 10, 2025

Abstract

Shojaee et al. (2025) report that Large Reasoning Models (LRMs) exhibit "accuracy collapse" on planning puzzles beyond certain complexity thresholds. We demonstrate that their findings primarily reflect experimental design limitations rather than fundamental reasoning failures. Our analysis reveals three critical issues: (1) Tower of Hanoi experiments systematically exceed model output token limits at reported failure points, with models explicitly acknowledging these constraints in their outputs; (2) The authors' automated evaluation framework fails to distinguish between reasoning failures and practical constraints, leading to misclassification of model capabilities; (3) Most concerningly, their River Crossing benchmarks include mathematically impossible instances for $N \geq 6$ due to insufficient boat capacity, yet models are scored as failures for not solving these unsolvable problems. When we control for these experimental artifacts, by requesting generating functions instead of exhaustive move lists, preliminary experiments across multiple models indicate high accuracy on Tower of Hanoi instances previously reported as complete failures. These findings highlight the importance of careful experimental design when evaluating AI reasoning capabilities.

Comment on The Illusion of Thinking

Takeaways



An overview of the LLM development and training pipeline, with a focus on new pre-training and post-training methodologies discussed in this article

Sebastian Raschka, 2024, New LLM Pre-training and Post-training Paradigms [Blog]

References

Papers:
The Illusion of Thinking
Comment on The Illusion of Thinking

Blogs:
O1 System Card
Introducing OpenAI o3 and o4-mini

Youtube videos:
Reinforcement Learning from Human Feedbacks
Understand Proximal Policy Optimization